Deliverable 2 ? Initial Implementation Report

Course: CY321 ? Secure Software Design with Secure Coding Implementation
Deliverable: D2 ? Initial Implementation Phase
Project: AUI Secure ? DevSecOps CI/CD Pipeline & Secure Coding Demonstration Platform
Submission Date: 19/04/2026
Report Date: 2026-05-07

1. What Deliverable 2 Required

Deliverable 2 is the initial working implementation of the system. It must show that the security requirements from D1 have been translated into real, running code. The key requirement is a functional application with the core security controls in place, even if not fully hardened yet.

The D2 snapshot lives in Deliverable-2/ which contains:

- The Flask application (app/ with all modules)
- The full test suite (tests/)
- The CI/CD pipeline (.github/workflows/devsecops.yml)
- Pre-commit hooks (.pre-commit-config.yaml)
- Auto-remediation script (remediate.py)

2. What Was Built by D2

2.1 Application Structure at D2

At the time of D2 submission, the following modules were complete and working:

Module	Purpose	Security Features Present
app/main.py	App factory	Session expiry, error handlers (no stack trace leakage)
app/config.py	Configuration	All settings from env vars, RuntimeWarning on missing SECRET_KEY
app/db.py	Database	6-table schema, FK enforcement, per-request connections
app/auth.py	Authentication	scrypt hashing, lockout, session rotation, CSRF
app/rbac.py	Access control	@requires_auth, @requires_role decorators
app/security.py	Security middleware	Rate limiter, 7 security headers, CSP nonce,
app/audit.py	Audit logging	log_action() on every action, append-only
app/validators.py	Input validation	Username regex, password policy, bleach sanitization
app/api_keys.py	API keys	SHA-256 hashing, Bearer auth, soft revocation
app/dashboard.py	Admin panel	@requires_role("admin") on all routes
app/items.py	CRUD + search	WHERE user_id = ?, parameterised LIKE, insecure endpoint gated
app/lab.py	Security labs	Attack/Defend endpoints for SQLi, XSS, hashing, headers

app/profiles.py	IDOR demo	Attack endpoint + secure /users/me

2.2 D2 Test Results

At submission, all tests were passing:

```
pytest tests/ -v
62 passed
```

The test suite covered:

- 22 functional tests (auth flows, CRUD, headers, user isolation)
- 40 security tests (SQLi payloads, XSS payloads, lockout, RBAC, CSRF, API keys)

---

3. D1 Requirements ? D2 Implementation Mapping

3.1 Authentication (SR-01, SR-02, SR-07, SR-10)

What D1 required:

- Passwords stored using strong hashing
- Account lockout after failed attempts
- Session cookies with HttpOnly and SameSite flags
- Rate limiting on authentication endpoints

What D2 implemented:

Password hashing (app/auth.py:register):

```
password_hash = generate_password_hash(password)
# Uses scrypt: memory-hard, GPU-resistant, random salt per hash
```

Account lockout (app/auth.py:_record_failure):

```
secs = min(base_duration * (2 ** full_lockouts), 86_400)
locked_until = (now + timedelta(seconds=secs)).isoformat()
```

Session cookie flags (app/config.py):

```
self.SESSION_COOKIE_HTTPONLY = True
self.SESSION_COOKIE_SAMESITE = "Lax"
self.SESSION_COOKIE_SECURE = os.environ.get("FLASK_ENV") == "production"
```

Rate limiting (app/auth.py):

```
@bp.post("/login")
@limiter.limit("10 per minute")
def login():
```

Verification: test_account_lockout_after_max_attempts and test_password_policy pass in the D2 test suite.

3.2 Authorisation (SR-03, SR-04)

What D1 required:

- Users can only access their own data
- Admin routes restricted to admin role

What D2 implemented:

User data isolation (app/items.py):

```
def _uid():
    return session.get("user_id") or getattr(g, "api_user", {}).get("id")

# Every query uses _uid() ? user can never see other's items
rows = db.execute("SELECT * FROM items WHERE user_id = ?", (_uid(),))
```

Admin restriction (app/rbac.py):

```
@requires_role("admin")
def get_security_stats():
    # Only reachable with role='admin' in server-side session
```

Verification: test_delete_other_users_item_fails and test_non_admin_dashboard_blocked pass.

3.3 Input Validation (SR-05)

What D1 required:

- All user input validated before use
- HTML stripped to prevent XSS storage
- Username and password policies enforced

What D2 implemented:

Username validation (app/validators.py):

```
_USERNAME_RE = re.compile(r"^[a-zA-Z0-9_]{3,32}$")
# Blocks: spaces, @, ../etc/passwd, Unicode confusables
```

Password policy (app/validators.py):

```
def validate_password(password):
    if len(password) < 8: raise ValidationError(...)
    if not re.search(r"[A-Z]", password): raise ValidationError(...)
    if not re.search(r"[a-z]", password): raise ValidationError(...)
    if not re.search(r"\d", password): raise ValidationError(...)
    if not re.search(r"[@$!%*?&]", password): raise ValidationError(...)
```

HTML sanitisation (app/validators.py):

```
def sanitize_text(text, max_length=1000):
    cleaned = bleach.clean(text, tags=[], strip=True)
    return cleaned.replace("\x00", "")[:max_length]
```

Verification: test_password_policy (6 cases), test_invalid_usernames_rejected (5 cases), test_xss_payloads_stripped_from_title (3 cases) all pass.

3.4 SQL Injection Prevention (SR-06)

What D1 required:

- All database queries use parameterised statements

What D2 implemented:

Parameterised queries everywhere on the secure path:

```
# auth.py ? login query
row = db.execute(
    "SELECT id, username, password_hash FROM users WHERE username = ?",
    (username,)
).fetchone()

# items.py ? secure search
rows = db.execute(
    "SELECT * FROM items WHERE user_id = ? AND title LIKE ?",
    (_uid(), f"%{q}%")
).fetchall()

# api_keys.py ? key lookup
row = db.execute(
    "SELECT u.id, u.username, u.role FROM api_keys k "
    "JOIN users u ON k.user_id = u.id WHERE k.key_hash = ?",
    (key_hash,)
).fetchone()
```

The insecure endpoint for demo purposes is gated by SECURE_MODE and clearly marked VULNERABLE:

```
@bp.post("/sqli/attack")
def sqli_attack():
    if current_app.config.get("SECURE_MODE", True):
        return jsonify({"blocked": True}), 403
    sql = f"SELECT ... WHERE name LIKE '%{q}%'"  # INTENTIONALLY VULNERABLE
```

Verification: test_secure_search_resists_sql_injection with 6 payloads passes. All return 0 results, not crashes.

3.5 CSRF Protection (SR-08)

What D1 required:

- CSRF tokens must protect state-changing requests

What D2 implemented:

```
# security.py ? token generation
def generate_csrf_token():
    token = secrets.token_hex(32)  # 256 bits
    session["csrf_token"] = token
    return token

# security.py ? token validation
def csrf_protect(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        incoming = _read_csrf_from_request()
        stored = session.get("csrf_token", "")
        if not secrets.compare_digest(incoming, stored):
            return jsonify({"error": "CSRF token missing or invalid"}), 403
        return f(*args, **kwargs)
```

Verification: test_csrf_token_returned_on_login passes.

3.6 Security Headers (SR-09)

What D1 required:

- Security headers on every response

What D2 implemented (app/security.py:_apply_security_headers):

All 7 headers applied via @app.after_request:

- Content-Security-Policy ? XSS prevention
- X-Frame-Options: DENY ? clickjacking prevention
- X-Content-Type-Options: nosniff ? MIME confusion prevention
- X-XSS-Protection: 1; mode=block ? legacy browser XSS filter
- Referrer-Policy: strict-origin-when-cross-origin ? info leakage prevention
- Permissions-Policy: geolocation=(), camera=(), microphone=() ? sensor API blocking
- Strict-Transport-Security ? HTTPS enforcement (production only)

Verification: test_security_headers_present passes ? checks CSP, X-Frame-Options, X-Content-Type-Options on every response.

3.7 Audit Logging (SR-11)

What D1 required:

- All security-relevant actions must be logged

What D2 implemented (app/audit.py):

```
def log_action(action, resource=None, details=None, success=True, ...):
    db.execute(
        "INSERT INTO audit_logs (user_id, username, action, resource, "
        "ip_address, user_agent, details, timestamp, success) VALUES (?,?,?,?,?,?,?,?,?)",
        (uid, uname, action, resource, ip, ua, details_json, now, int(success))
    )
```

Called by every module:

- auth.py: LOGIN_SUCCESS, LOGIN_FAILED, LOGIN_BLOCKED, LOGOUT, USER_REGISTER, PASSWORD_CHANGED
- items.py: item create/update/delete actions
- lab.py: LAB_SQLI_ATTACK, LAB_SQLI_DEFEND, LAB_XSS_ATTACK, LAB_XSS_DEFEND, etc.
- rbac.py: ACCESS_DENIED
- dashboard.py: user management actions

3.8 Dependency and Secret Scanning (SR-12, SR-13)

What D1 required:

- Scan dependencies for CVEs
- Scan code for hardcoded secrets

What D2 implemented:

CI pipeline Stage 2 (gitleaks) ? scans full git history:

```
- uses: gitleaks/gitleaks-action@v2
  with:
    fetch-depth: 0  # scans complete git history
```

CI pipeline Stage 4 (pip-audit) + auto-remediation:

```
  - run: pip-audit -r requirements.txt --format json -o pip-audit.json
  - run: python remediate.py  # auto-bumps versions if CVE found
  - run: gh pr create --label security  # opens a PR for review
```

Result at D2: No secrets in history. No CVEs in dependencies.

---

4. The CI/CD Pipeline at D2

The full 5-stage pipeline was operational at D2 submission:

```
  Stage 1: build-and-test     pytest 62 tests
  Stage 2: secret-scan        gitleaks v8.18 full history scan
  Stage 3: sast               bandit -r app/ -ll -ii
  Stage 4: sca                pip-audit + remediate.py + auto-PR
  Stage 5: dast               OWASP ZAP baseline scan (push to main only)
```

Pre-commit hooks also running locally:

- black (formatting)
- bandit (SAST)
- pip-audit (SCA)
- gitleaks (secret scan)
- YAML/JSON validity, private key detection, large file blocking

---

5. Security Issues Present in D2 (Fixed in D3)

Although D2 met the core requirements, the Deliverable-3 review process identified 6 issues that were present in the D2 code. These are documented in full in Deliverable-3/Security_Fixes_Changelog.md.

ID	Severity	File	Issue in D2
F1	HIGH	profiles.py	IDOR attack endpoint NOT gated by SECURE_MODE ? would ship live in production
F2	Medium	auth.py	time.sleep(0.5 * attempts) blocked worker thread ? DoS amplifier
F3	Medium	lab.py	SECURE_MODE toggle was @requires_auth only ? any user could disable security
F4	Low	items.py	from .validators import ... placed after a function definition
F5	Low	db.py	No admin demo account ? admin code paths untestable
F6	Low	security.py	CSP nonce silently broke all ~70 inline onclick= handlers in the SPA

F1 was the most critical. The combination of F1 and F3 meant:

Any logged-in user could POST to /lab/toggle-secure-mode to set SECURE_MODE=false
Then GET /users/attack/' OR '1'='1 to dump all users including admin password hash

This was caught and fixed before the D3 submission.

6. D2 vs D1 Requirements ? Compliance Table

D1 Requirement	D2 Status	Evidence
SR-01 Passwords hashed	COMPLETE	scrypt in auth.py; test_password_policy passes
SR-02 Account lockout	COMPLETE	exponential backoff in auth.py; test_account_lockout passes
SR-03 User data isolation	COMPLETE	WHERE user_id = ? in items.py; test_delete_other_users_item_fails
SR-04 Admin routes restricted	COMPLETE	@requires_role("admin") in rbac.py; test_non_admin_dashboard_blocked
SR-05 Input validation	COMPLETE	validators.py at all endpoints; test_invalid_usernames_rejected
SR-06 Parameterised SQL	COMPLETE	All secure-path queries use ?; test_secure_search_resists_sql_injection
SR-07 Secure cookies	COMPLETE	HttpOnly + SameSite=Lax in config.py
SR-08 CSRF protection	COMPLETE	csrf_protect in security.py; test_csrf_token_returned_on_login
SR-09 Security headers	COMPLETE	7 headers in security.py; test_security_headers_present
SR-10 Rate limiting	COMPLETE	Flask-Limiter in security.py and auth.py
SR-11 Audit logging	COMPLETE	audit.py called everywhere
SR-12 Dependency scanning	COMPLETE	pip-audit in CI Stage 4
SR-13 Secret scanning	COMPLETE	gitleaks in CI Stage 2
SR-14 API key hashing	COMPLETE	SHA-256 in api_keys.py

All 14 D1 security requirements were implemented by D2.

7. Summary

Deliverable 2 demonstrated that every security requirement from D1 was translated into working Python code and verified by automated tests. The application was functional, tested, and had a full CI/CD security pipeline. The 6 issues found during D3 review represent normal refinement ? they were caught by the same review process we advocated, and all were fixed before final submission.

D2 Verdict: All D1 requirements implemented. 62/62 tests passing. CI pipeline operational. 6 minor-to-high issues identified for D3 hardening.